```
import numpy as np
```

```
def sigmoid(x): #defining sigmoid activation function
    return 1/(1+np.exp(-x))
```

```
x=np.array([[1,0,0], #input data (with bias term in first column)
            [1,0,1],
            [1,1,0],
            [1,1,1]])
```

```
def relu(x): #return value of relu function
    return np.maximum(0,x)
```

```
#expected output (same output as XOR gate)
y=np.array([[0],
           [1],
           [1],
            [0]])
```

```
#initiaze random weights
np.random.seed(42) #for reproducibility
w1=np.random.randn(3,4)
print("w1:",w1)#weights of no. of input units->no. of hidden units
w2=np.random.randn(4,1)
print("\nw2:",w2)#weights of no. of hidden units->no. of output units
```

```
w1: [[ 0.49671415 -0.1382643   0.64768854  1.52302986]
 [-0.23415337 -0.23413696  1.57921282  0.76743473]
 [-0.46947439  0.54256004 -0.46341769 -0.46572975]]

w2: [[ 0.24196227]
 [-1.91328024]
 [-1.72491783]
 [-0.56228753]]
```

```
#----forward pass----
z1=np.dot(x,w1)
print("hin:\n\n",z1) #input layer
a1=sigmoid(z1)
print("\nhout:\n\n",a1)
z2=np.dot(a1,w2)
print("\nYin:\n\n",z2)#yin
y_pred=sigmoid(z2)
```

```
hin:

 [[ 0.49671415 -0.1382643   0.64768854  1.52302986]
 [ 0.02723977  0.40429574  0.18427085  1.0573001 ]
 [ 0.26256078 -0.37240126  2.22690135  2.29046459]
 [-0.20691361  0.17015879  1.76348366  1.82473483]]

hout:

 [[0.62168683 0.46548889 0.65648939 0.82098421]
 [0.50680952 0.59971932 0.5459378  0.74217425]
 [0.56526568 0.40796092 0.90263938 0.90808424]
 [0.44845537 0.54243735 0.85364543 0.8611333 ]]

Yin:

 [[-2.33420537]
 [-2.38381551]
 [-2.71135381]
 [-2.88599813]]
```

```
#print predictions
print("Forward pass predictions (before training) Yout :")
print("\n",y_pred)
print("\nFound out that y_pred is different from actual y:\n\n",y)
```

```
Forward pass predictions (before training) Yout :

 [[0.08832943]
 [0.0844152 ]
```

```
 [0.06230671]
 [0.05285008]]

Found out that y_pred is different from actual y:

 [[0]
 [1]
 [1]
 [0]]
```

```
mse=np.mean((y-y_pred)**2)
print("Mean Squared Error:",mse)
```

```
Mean Squared Error: 0.43203986372326847
```

```
Start coding or generate with AI.
```